# Estructura y funcionamiento

Aplicación web de gestión para el sistema de clínicas

## Organización del proyecto

```
frontend/src/app/
├── core/  auth, guards e interceptor HTTP
├── features/
|   ├── auth/  inicio de sesión
|   └── pacientes/  modelo, servicio y página
├── layout/  navegación y sesión
├── shared/  componentes reutilizables
└── app.routes.ts  definición de rutas
```

## Models, Services y Pages

El modelo Paciente define la forma de los DTO que entrega Spring Boot. PacienteService concentra las llamadas HTTP para listar, crear y eliminar. PacientesComponent muestra esos datos y envía acciones al servicio. Los componentes no contienen registros de pacientes codificados.

## Consumo del backend Spring Boot

La API clínica publica GET y POST /api/paciente, además de DELETE /api/paciente/{id}. El proxy de Angular reenvía /api/* a http://localhost:8080. El servicio de autenticación escucha en 8081; /auth/api/auth/login se reenvía allí y se transforma a /api/auth/login.

## Inicio de sesión y JWT

AuthService envía usuario y contraseña a POST /api/auth/login. La respuesta incluye accessToken, tokenType y expiración. El token se guarda en localStorage junto al nombre y los roles extraídos de scope. authInterceptor añade Authorization: Bearer <token> a las solicitudes HTTP.

# Accesos y protección de rutas

## Menú según permisos

El menú lee los roles de la sesión autenticada. ROLE_USER ve Pacientes. ROLE_ADMIN ve Pacientes y Administración. Las cuentas creadas por el endpoint de registro reciben ROLE_USER; un operador debe asignar ROLE_ADMIN mediante la base auth_db para habilitar la segunda opción.

| Rol | Pacientes | Administración |
|---|---|---|
| ROLE_USER | Visible | Oculta |
| ROLE_ADMIN | Visible | Visible |

## Guards de navegación

authGuard exige una sesión con token y redirige a /login si falta. roleGuard compara el claim de roles con data.roles configurado en la ruta. /pacientes permite ROLE_USER y ROLE_ADMIN; /administracion exige ROLE_ADMIN. Si no coincide, la aplicación presenta Acceso restringido.

## Ejecución

Con Java 21, MySQL y Node.js instalados, prepara las bases y llaves indicadas en AUTHENTICATION.md. Inicia primero auth-service en puerto 8081 y después la API clínica en 8080. Desde frontend ejecuta npm install y npm start; abre http://localhost:4200.

## Evidencia de usuarios

La matriz anterior documenta las opciones previstas para ambos roles. Las capturas de dos sesiones reales deben tomarse en un entorno con los dos servicios y usuarios provisionados; este repositorio no contiene cuentas de prueba ni una base de datos con usuarios, y el entorno de generación no pudo instalar las dependencias por la restricción de red. No se presentan imágenes simuladas como capturas de funcionamiento.

# Búsqueda, filtros y paginación

## Flujo de la consulta

PacienteFilter recoge nombre, apellido, DPI, activo, page y size. PacienteSpecification compone predicados opcionales de Specification<Paciente>; los textos se buscan por coincidencia parcial ignorando mayúsculas y varios criterios se combinan con AND.

PacienteRepository extiende JpaSpecificationExecutor, por lo que recibe la Specification y Pageable. El servicio construye PageRequest con page, size y Sort id DESC. Spring ejecuta en la base de datos el WHERE filtrado y la selección ordenada de la página, además del conteo. No se llama findAll() sin límites ni se filtra una lista en memoria.

## Page y metadata

Page<PacienteDto> ofrece getContent(), getTotalElements(), getTotalPages(), getNumber(), getSize(), hasNext() y hasPrevious(). El controlador entrega la lista en data y los campos totalRecords, page, pageSize, totalPages, hasPreviousPage y hasNextPage en metadata.

totalPages es ceil(totalRecords / pageSize). Por ejemplo, 125 coincidencias con tamaño 50 generan 3 páginas: 50, 50 y 25 elementos. Si hay 50,000 filas y size=50, una petición transporta como máximo 50 pacientes más la metadata; el conteo informa cuántos cumplen los filtros.

## Interacción en Angular

La pantalla envía los filtros vigentes junto con page y size. Buscar reinicia page a 0; Anterior/Siguiente solicitan una nueva página por HTTP y se habilitan con las banderas de metadata. El selector ofrece 50, 100 y 200 registros y también vuelve a la primera página al cambiar.

## Demostración en Network o Postman

Con el usuario autenticado, observa GET /api/pacientes?page=0&size=50, luego page=1 y page=2. Cada respuesta incluye hasta 50 elementos, páginas con registros diferentes, el totalRecords y las banderas prev/next. Combina después ?nombre=Maria&activo=true&page=0&size=50: totalRecords cuenta solo las coincidencias filtradas. Los tamaños permitidos por la API son 50, 100 y 200; page parte en 0 y un tamaño no permitido responde 400.

# Conceptos clave y evidencia

## Diferencias y responsabilidades

Paginación divide el conjunto en fragmentos solicitados. En cliente se descarga todo y solo se separa visualmente; en servidor cada petición recupera el fragmento necesario y reduce transferencia y memoria. Pageable describe página, tamaño y orden; PageRequest implementa ese contrato. Specification construye filtros componibles; JpaSpecificationExecutor ejecuta esos filtros con JPA. Sort determina un orden estable antes del corte en páginas. List no lleva conteo ni navegación. El máximo de size protege base de datos, red y memoria frente a solicitudes excesivas.

## Evidencia sugerida en DevTools Network

1. Inicia sesión, abre la pantalla de Pacientes y selecciona la petición GET /api/pacientes?page=0&size=50. En Preview confirma metadata.page=0 y hasta 50 elementos en data. 2. Pulsa Siguiente dos veces; registra page=1 y page=2. Comprueba que hasPreviousPage cambia a true después de la primera página y que hasNextPage refleja si existe otra página. Las páginas tienen resultados distintos por el orden id DESC.

3. Aplica nombre=Maria y estado=Activo. La URL combina nombre=Maria&activo=true&page=0&size=50. Verifica que totalRecords cuenta solo esas coincidencias. 4. Cambia el tamaño a 100 y verifica size=100 y page=0. Estos datos de Network y las respuestas JSON pueden usarse como capturas propias de la demostración.

## Peticiones Postman

GET /api/pacientes?page=0&size=50 | page=1&size=50 | page=2&size=50 | /api/pacientes?nombre=Maria&activo=true&page=0&size=50. Incluye Authorization: Bearer <JWT> en cada petición. Configuración de servicios: AUTHENTICATION.md; ejemplos: API.md.

## Nota de preparación

Para producir evidencia de dos usuarios con permisos distintos, el entorno debe tener datos de pacientes y ambos servicios en ejecución. Las cuentas se autentican con auth-service; el rol ROLE_ADMIN debe provisionarlo un operador en auth_db. Esta guía describe las comprobaciones a capturar y no inventa resultados de una base de datos que no está disponible en este documento.